

HelixCluster

HelixCluster

AI hesaplamaları için dağıtık bir işletim sistemi – veri merkezi GPU’larından uçtaki el cihazlarına, tek bir kontrol düzlemi altında.

Özet

Helix Küme İşletim Sistemi, yeni nesil bir dağıtık işletim sistemi olup, veri merkezi GPU’larından uçtaki tek kartlı bilgisayarlara (SBC) ve el cihazlarına kadar heterojen düğümler arasında hesaplama işlemlerini düzenler. Yüksek performanslı hesaplama (HPC) zamanlamasını, konteyner orkestrasyonunu, AI/ML çıkarımını, çoklu küme federasyonunu ve güvenli çok kiracılı oturumları tek bir kontrol düzlemi altında birleştirir.

Kısa açıklama

Go tabanlı dağıtık bir işletim sistemi / GPU paylaşımlı hesaplama kümesi. HPC zamanlamasını (Omega modeli iki seviyeli zamanlayıcı), konteyner orkestrasyonunu, AI çıkarım yönlendirmesini, federasyonu ve heterojen düğümler arasında güvenli çok kiracılı oturumları SWIM dedikodu protokolü ve Raft konsensüsü ile koordine ederek birleştirir. Kuantum sonrası uçtan uca şifreleme ile donatılmıştır.

Uzun açıklama

Helix Küme İşletim Sistemi, veri merkezi GPU’larından uçtaki tek kartlı bilgisayarlara, hatta el cihazlarına kadar son derece heterojen donanımlar üzerinde hesaplama iş yüklerini tek bir kontrol düzlemi altında düzenler. Bir A100 rafını ve bir avuç SBC’yi bir düzine uyumsuz ada yerine tek bir adreslenebilir ağ olarak ele alır. Go çalışma alanı (monorepo artı git alt modülleri) olarak uygulanan yedi katmanlı bir yığın sunar; L0 donanım katmanından L7 federasyon ve gözlemlenebilirliğe kadar uzanır ve

on dört kontrol düzlemi mikro hizmeti tarafından koordine edilir. Düğüm üyeliği, düğümlerin katılım ve ayrılışlarına göre ağın kendini onarmasını sağlayan SWIM dedikodu ve keşif protokolü ile izlenir. Güçlü tutarlılığa sahip durumlar, hız ve STONITH çitleme için kiralayan-yerel okumalarla düzenlenen Raft konsensüsü grupları üzerinden yönetilir; böylece bölünmüş bir düğüm paylaşılan durumu bozamaz. İş yükü yerleştirme, Omega modeli iki seviyeli bir zamanlayıcı aracılığıyla gerçekleşir — iyimser eşzamanlılık, ClassAd eşleştirmesi, grup zamanlaması, değer çarpanı önceliği ve kısıtlara dayalı yerleştirme — ve klasik bir HPC zamanlayıcısının ötesine geçer: karbon farkındalığı, maliyet/TCO farkındalığı, buluta ani ölçeklendirme ve piyasa adaptörleri (Akash, io.net, RunPod, AWS Spot, Chutes) sayesinde yerel kaynaklar tükendiğinde iş yükü kiralık kapasiteye taşınabilir.

Son kullanıcılar bu mekanizmayı doğrudan görmez; onlar, temiz bir oturum modeli (hesaplama tahsisleri), etkileşimli bir WebSocket/PTY terminali, dahili bir AI çıkarım rotası ve havuz kullanım okumaları aracılığıyla etkileşimde bulunur. Güvenlik, sonradan eklenen bir özellik değil, temel bir katmandır: SPIFFE kimliği, cihaz doğrulaması (meydan okuma/yanıt, GPU iş kanıtı, mühürleme), ihracat kontrolü KYC kapısı ve X25519 + ML-KEM-768 hibrit anahtar değişimi üzerine inşa edilmiş, AEAD kayıt koruması ve tekrar saldırısı reddi ile kuantum sonrası uçtan uca şifreli bir taşıma katmanı — bugün ele geçirilen trafiğin, yarının kuantum tehditlerine karşı bile gizli kalmasını sağlayacak şekilde tasarlanmıştır. Doğruluk iddia edilmez, *kanıtlanır*: Deterministik simülasyon testleri (FoundationDB tarzı tohumlanmış çalıştırmalar, hata enjeksiyonu, ağ simülasyonu, bayt bayt tekrar oynatma ve Porcupine doğrusallık denetleyicisi) dağıtık arızaları talep üzerine yeniden üretir; zorunlu eşleştirilmiş mutasyon testleri, koruma testlerinin gerçekten işe yaradığını kanıtlar. Mimari ve belgeler, gerçeklik ile dokümantasyon arasındaki uyumsuzluk anında derlemeyi başarısız kılan mekanik denetimlerle dürüst tutulur.

Neden inşa ettik

AI ve HPC iş yüklerini, birbirinden tamamen farklı donanım katmanlarında çalıştırmak için ayrı ayrı zamanlayıcılar, orkestratörler ve çıkarım yığınlarını bir araya getirmeye gerek kalmadan — üstelik her gönderilen özelliğin *gerçek son kullanıcı davranışını* kanıtlamasını (asla sahte testler yerine yeşil testler değil) ve her işletim sistemi özelindeki yeteneğin, platforma özgü gerçek bir yerel altyapı kullanmasını (Linux’a özel taklitler yok) mühendislik garantisiyle sağlamak için inşa ettik. Depodaki yönetim belgesinde alıntılanan temel sorun, “testler geçiyor ama özellik aslında çalışmıyor” başarısızlık modudur; proje, bu sorunu ortadan kaldırmak üzere açıkça tasarlanmıştır.

Neden oyunun kurallarını deęiřtiriyor

Normalde beř ayrı yığın olan — HPC zamanlaması, konteyner orkestrasyonu, AI çıkarımı, çoklu küme federasyonu ve güvenli çok kiracılı oturumlar — beř unsuru, veri merkezi GPU'larından kenar cihazlardaki elde taşınabilir cihazlara kadar uzanan tek bir kontrol düzleminde birleřtiriyor. Üstelik bunu, genellikle yalnızca özel altyapılara ayrılan bir titizlik bütçesiyle yapıyor: Biçimsel yöntemler düzeyinde doğruluk (TLA+ spesifikasyonları, deterministik simülasyon, doğrusallaştırılabilirlik kontrolü) ve kuantum sonrası gizli taşıma gibi garantiler, çoęu orkestratörün basitçe denemeye bile cesaret edemedięi türden. Teknik farklılıkların ötesinde, maliyet ve karbon farkındalıęına sahip yerleřtirme ile bulut pazarı patlaması, onu aynı zamanda *ekonomik* bir kaldıraç haline getiriyor — zamanlayıcı, daha ucuz, daha yeřil veya boş kapasiteyi otomatik olarak takip edebiliyor; böylece aynı iş yükü, kimsenin işi yeniden yazmasına gerek kalmadan daha az maliyetle ve daha az emisyonla çalıştırılabilir.

Yenilikçi olan ne

- **Deterministik Simülasyon Testi (DST)** — Hatalar, saat kaymaları ve ağ bölünmeleri enjekte eden, bunları bayt bayt yeniden oynatan ve sonucu bir Porcupine doğrusallaştırılabilirlik kontrolünden geçiren, tohumlanmış ve tamamen yeniden üretilebilir bir simülatör. Böylece bir Heisenbug bir kez yakalandığında, sonsuza dek isteęe baęlı olarak yeniden üretilebilir.
- **Omega-modeli iki seviyeli zamanlayıcı** — İyimser eşzamanlılık yerleřtirmesi, ClassAd eşleřtirmesi, grup zamanlaması ve deęer çarpanı öncelięi ile, birçok zamanlayıcının merkezi bir darboęaz olmadan tek bir kümeye karşı taahhütte bulunmasını saęlayan paylaşılan durum tasarımı.
- **Kuantum sonrası Uçtan Uca Şifreleme / gizli çıkarım altyapısı** — X25519 + ML-KEM-768 hibrit anahtar deęiřimi, her istek için yanıt anahtar çifti baęlama ve tekrarlama reddi ile AEAD (kripto ilkeleri gerçek ve test edilmiř; tam gizli çok düęümlü tur hâlâ açıkça PLANLANMIř/kapalı).
- **Kanıt odaklı güven** — Düęümler ne olduklarını *kanıtlamak* zorunda: SPIFFE kimlięi, GPU iş kanıtı, cihaz mühürleme, ihracat kontrolü KYC kapısı ve AB AI Yasası uyumluluk dökümantasyonu üretimi. Böylece güven, ağ konumuna göre varsayılmak yerine kanıtlarla kazanılır.
- **Maliyet ve karbon farkındalıęına sahip orkestrasyon** — Toplam sahip olma maliyeti modellemesi, karbon farkındalıęına sahip yerleřtirme, buluta ani yük artışı, N+K yedeklilik rezervi ve bulut pazarı adaptörleri. Böylece fiyat ve emisyonlar, sonradan akla gelenler deęil, zamanlama sürecinin birinci sınıf girdileri haline gelir.

- **Çoklu Raft konsensüsü** — Düşük gecikmeli tutarlılık için kiralık yerel okumalarla desteklenen, her parçaya özel Raft grupları. STONITH çitleme (IPMI / EC2 / Azure / SBD) ile takılı kalan bir düğüm kesin olarak kaldırılır, durumu bozmak üzere bırakılmaz.
- **Sürüklenme karşıtı mekanik lintler** — archlint, belgelenen bir bileşenin var olmayan bir paket yoluna eşleştiği anda derlemeyi başarısız kılar. Bir dökümantasyon zinciri motoru da Markdown / HTML / PDF / DOCX dosyalarını bayt bazında tutarlı tutar. Böylece belgeler, kod hakkında sessizce yalan söyleyemez.

İçerik

En Büyük Teknik Zorluklar ve Çözümlerimiz

- **“PASS-blöf” (testler işlevsel olmayan özelliklerde bile geçiyor).** Tüm projenin ortadan kaldırmak için inşa edildiği hata türü: sahte kodlar üzerinden yeşil test paketi. Zorunlu eşli mutasyon testi ile çözüldü — her iş ögesi, tamamlanmadan önce bağımsız bir kod mutasyonu altında *başarısız olması gereken* bir koruma testi ile birlikte gelir; böylece geçen bir testin gerçek davranışı, bir sahte kodu değil, kanıtlanmış şekilde çalıştırdığı garanti edilir.
- **Çapraz platform uyumu (yalnızca Linux’a özgü sahte kodlar yok).** Derleme etiketleriyle bölünmüş ortak bir arayüzle çözüldü — Linux cgroup / /proc / çekirdek WireGuard, macOS sysctl / vm_stat / IOKit / wireguard-go — ve ardından bağımsız bir işletim sistemi doğrulayıcısıyla çapraz kontrol yapıldı; böylece her platform, Linux kurgusu yerine gerçek yerel durumu raporlar.
- **Dağıtık sistemlerde hata durumunda doğruluk.** Deterministik simülasyon testi ve doğrusallık denetleyicisi ile çözüldü; bu sistem, bölünmeleri, çökmeleri ve saat kaymalarını üreterek yeniden oynatır ve TLA+ biçimsel şartnameleriyle desteklenir; bu şartnameler, tek bir kod satırı çalıştırılmadan önce konsensüs ve zamanlama değişmezlerini sabitler.
- **Belgelendirme ve mimari sapması.** archlint ile çözüldü; bu araç, belgelenmiş ancak var olmayan bir paket eşlemesi tespit edildiğinde derlemeyi başarısız kılar. Ayrıca, kaçış yolu olmayan bir docs_chain doğrulama kapısı bulunur — sapma, eski bir wiki sayfası değil, derleme hatasıdır.
- **Tamamlanmamış işlerin dürüstçe kapsamlandırılması.** Gizli çok düğümlü çıkarım turu, kasıtlı olarak bir biletin arkasına alındı ve “henüz uçtan uca doğrulanmadı” olarak etiketlendi; böylece tamamlanmamış işler, tamamlanmış olanlar kadar disiplinli bir şekilde ele alındı.

Teknoloji Yığını

- **Go (go.mod: 1.25 / araç zinciri 1.26.4)** — yaklaşık 30 modüllük bir çalışma alanında kontrol düzlemi dili; veri merkezinden uç noktalara kadar aynı şekilde dağıtılabilen ucuz goroutine eşzamanlılığı ve statik ikili dosyalar için seçildi.
- **Zig (0.14+) + C/C++** — Go'un çalışma zamanı engel oluşturduğu durumlarda tercih edildi: donanım üzerinde deterministik, tahsisatsız kontrol gerektiren düşük seviyeli sistem ilkeleri ve GPU çekirdekleri.
- **gRPC + Protocol Buffers** — her alt sistem arasındaki API (api/v1/) tip güvenli, sürümlü bir sözleşmedir; böylece on dört mikro hizmet, birbirlerini bozmadan veya elle yazılmış veri biçimlerine başvurmadan evrilebilir.
- **Raft (etcd-raft) + SWIM dedikodusu** — kasıtlı bir ayırım: Raft, *kesinlikle güçlü tutarlılık gerektiren* durumu taşıırken, SWIM dedikodusu ölçeklenebilir üyelik ve keşif işlemlerini, konsensüsün çok ağır olacağı durumlarda yönetir.
- **PostgreSQL 16, Redis 7 kümesi, etcd v3.5, SQLite** — her iş için doğru depolama: ilişkisel kalıcı durum için PostgreSQL, sıcak önbellek için Redis, koordinasyon için etcd ve düğüm yerel HXC iş ögesi kaydı için gömülü SQLite.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — üç farklı trafik türü için üç mesajlaşma omurgası: hızlı iç olaylar için NATS/JetStream, yüksek verimli dayanıklı akışlar için Kafka, klasik aracı semantikleri için RabbitMQ.
- **WireGuard ağı + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — düğümler arası hafif bir ağ için WireGuard, hibrit kuantum sonrası el sıkışma ve AEAD kayıtlarıyla sarıldı; böylece iletim, klasik ve kuantum saldırılarına karşı gizli kalır.
- **SPIFFE + JWT (HS256) + kapsam tabanlı RBAC + OPA** — katmanlı kimlik ve yetkilendirme: SPIFFE iş yükü kimliği için, JWT jetonlar için, kapsam tabanlı RBAC genel erişim için, OPA ise ince ayarlı politikaları kod olarak ifade etmek için kullanılır.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C izleme** — W3C bağlam yayılımı ile metrikler, kontrol panelleri ve dağıtık izleme; böylece bir istek, hizmetler ve donanım katmanları arasında takip edilebilir.
- **HashiCorp Vault 1.16** — gizli bilgiler ve anahtar materyalleri kod ve yapılandırmadan uzak tutulur ve denetim altında dağıtılır.
- **Docker Compose, Kubernetes (kustomize, sertleştirilmiş securityContext), Helm** — yerel başlatma için Compose, gerçek dağıtımlar için Kubernetes/Helm ve sertleştirilmiş güvenlik bağlamları; tek bir tanım, tüm ortamlara taşınır.
- **React + TypeScript + Vite (Node 20+)** — oturumlar, terminaller ve havuz kullanımı için hızlı, tip güvenli bir web arayüzü.

- **TLA+** — konsensüs ve zamanlama değişmezlerinin biçimsel şartnamesi; böylece en zor test edilebilir özellikler, uygulama öncesinde tasarım aşamasında kanıtlanır.

Durum ve dürüstlük notları

- **Durum: geliştirme aşamasında.** Bu sürüm erken aşamada (0.1.0-dev). Birkaç ileri düzey özellik — tam gizlilik sağlayan çok düğümlü çıkarım döngüsü, pazar yerleşimi ve doğrulama tabanlı zamanlama popülasyonu — depoda açıkça **PLANLANMIŞ / altyapıya bağlı** olarak işaretlenmiş olup, **tam olarak çalışır durumda** sunulmamaktadır. Kapsam rakamları kendi beyanına dayanmaktadır.
- **Lisans: Belirlenmedi.** Net bir şekilde ilan edilmemiştir; Helm şemasındaki HelixCluster/HelixCluster ve helixcluster.io URL'leri doğrulanmamış yer tutucular olup gerçek uzak depolarla eşleşmemektedir.
- Dahil edilen LLM yığını projeleri (LLMOrchestrator, LLMPProvider, LLMsVerifier), küme içinde barındırılan model sunucuları değil, birbirinden bağımsız alt modüllerdir.

Öncelik seviyesi: Helix-birincil (LLM-altyapı kümesi — çıkarım ve hesaplama iş yüklerini barındırabilen hesaplama altyapısı). HelixTrack'dan sonra gelir.